

2.2

the code without boilerplate is simple:

```
int main(){
    int v;
    printf("Enter the number of vertices in your graph:");
    scanf("%d", &v);

    Node** adj_list = (Node**)malloc((sizeof(Node**)) * v);
    if(adj_list == NULL){
        printf("Couldn't Allocate Mem...!");
        return 1;
    }

    for(size_t k =0; k<v; k++){
        Node* a = (Node* )malloc(sizeof(Node));
        if(a == NULL){
            printf("Couldn't Allocate Mem...!");
            return 1;
        }
        a->data = k+1;
        adj_list[k] = a;
    }
    while(true){

        int a,b;
        printf("Enter and edge in the graph from node 0 to node %d: ", v-
1);
        scanf("%d %d", &a, &b);

        if (a < 0 || b < 0 || b > v-1 || a > v-1 ) break;

        insert_end(adj_list[a], b+1);
        insert_end(adj_list[b], a+1);
    }

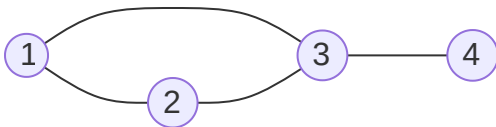
    for(size_t i=0; i<v; i++){
        print_list(adj_list[i]);
    }
    return 0;
}
```

ignoring all `malloc()` checks and print statements we get that the code works in a similar way like the adjacency matrix except in this case we already have a `Node** adj_list` that contains pointers to linked lists of all nodes to represent direct neighbors.

running the code yields:

```
A z .../graph_algos ? main ? v16.2.1 09:53 > ./a.out
Enter the number of vertices in your graph:4
Enter and edge in the graph from node 0 to node 3: 0 1
Enter and edge in the graph from node 0 to node 3: 1 2
Enter and edge in the graph from node 0 to node 3: 0 2
Enter and edge in the graph from node 0 to node 3: 2 3
Enter and edge in the graph from node 0 to node 3: -1 -1
1->2->3->
2->1->3->
3->2->1->4->
4->3->
```

we can see here that the graph represented is this one



we can say that we successfully implemented adjacency lists using Linked Lists.

2.3

a)

when it comes to the adjacency matrix the complexity calculation is simple we setup.

let $G = (V, E)$ be an undirected graph.

let $n = |V|$ and $A \in \mathcal{M}_n(\mathbb{R})$ represent the adjacency matrix

searching for an element requires knowing the exact vertex position, which can be accessed easily using the method: `adj_matrix[u][v]` thus:

search = $\mathcal{O}(1)$

for inserting or deleting an edge, we only need to update a single cell in the matrix.

iterating over all neighbors requires scanning an entire row of size V

for insertion we need to rescale and reallocate a bigger memory chunk, exactly $(V + 1)^2$
same for delete but in reverse, making it:

- search = $\mathcal{O}(1)$
- insert = $\mathcal{O}((V + 1)^2)$

- $\text{delete} = \mathcal{O}((V - 1)^2)$
- $\text{insert_edge} = \mathcal{O}(1)$
- $\text{delete_edge} = \mathcal{O}(1)$
- $\text{iterate} = \mathcal{O}(V)$

b)

assuming that we keep track of the last inserted node in each element of the adjacency list we can say that the complexity of insert is $\mathcal{O}(1)$ amortized.

removing a node requires us to go at worst through all nodes to remove that element. knowing that there are $|V|$ vertices we conclude that the delete operation is at worst a $\mathcal{O}(|V|)$

removing an edge (u,v) from our undirected graph, requires us to remove the edge from the list of neighbours in **u's** list and in **v's** list

NB: i just looked up the term degree of a vertex which means the number of edges to the concerned vertex ie **u's and v's list**. i will be using the known notation $\text{deg}(\cdot)$ from now on.

thus the delete operation has a complexity of $\mathcal{O}(\text{deg}(u) + \text{deg}(v))$.

searching for an edge (u,v) uses a similar though process.

we need to check if v is in **u's** the worst possible case is that v is in the tail of the linked list making the complexity $\mathcal{O}(\text{deg}(u))$

inserting an edge requires if we insert at end an amortized $\mathcal{O}(1)$.

same for iterating over all neighbors of v $\mathcal{O}(\text{deg}(v))$

thus:

- $\text{search} = \mathcal{O}(\text{deg}(u))$
- $\text{insert} = \mathcal{O}(1)$
- $\text{delete} = \mathcal{O}(|V|)$
- $\text{insert_edge} = \mathcal{O}(1)$
- $\text{delete_edge} = \mathcal{O}(\text{deg}(u) + \text{deg}(v))$
- $\text{iterate} = \mathcal{O}(\text{deg}(v))$